

## 实验三 跨平台应用开发实验报告

### 一、实验基本信息

实验名称\*\*：跨平台应用开发

实验编号\*\*：d20301035103、d20301009203

学时分配\*\*：4 学时

实验类型\*\*：验证型实验

每组人数\*\*：6 人

对应课程目标\*\*：课程目标 2

实验环境\*\*：Windows 11、HBuilderX、Visual Studio、Android Studio、Flutter SDK、Node.js、.NET SDK

### 二、实验目的

1. 掌握 Flutter、React Native、Uniapp、MAUI 四种主流跨平台框架的基础开发流程。
2. 实现跨平台应用的核心功能：RESTful API 网络请求、JSON 数据解析、列表展示与下拉刷新。
3. 学习移动设备传感器（GPS 定位、加速度计）的集成与调用方法。
4. 对比不同跨平台框架的开发效率、性能与适用场景，完成框架适用性分析报告。
5. 培养使用 AI 辅助工具提升编码效率的能力，掌握异常处理与测试验证方法。

### 三、实验原理与相关知识

#### （一）跨平台开发框架原理

跨平台框架基于\*\*一次编写、多端运行\*\*理念，通过中间层将代码编译/解析为对应平台原生代码，降低多端开发成本。

Flutter：基于 Dart 语言，采用自绘引擎，直接渲染 UI，性能接近原生。

React Native：基于 JavaScript 与 React，通过桥接调用原生组件，开发效率高。

Uniapp：基于 Vue 语法，编译生成多端代码，适配小程序与 App。

MAUI：基于 C#与.NET，统一多平台 API，原生性能与兼容性优秀。

#### （二）核心技术原理

1. RESTful API：基于 HTTP 协议的接口设计规范，通过 GET/POST 等方法获取与提交数据。

2. **\*\*JSON 解析\*\***: 将接口返回的 JSON 字符串转换为框架可识别的对象/数据结构。
3. **\*\*下拉刷新\*\***: 通过监听页面下拉事件, 重新请求接口并更新列表数据。
4. **\*\*设备传感器\*\***: 通过平台提供的 SDK/插件, 调用 GPS、加速度计等硬件获取设备状态数据。

### (三) AI 辅助编码

使用 TRAE 工具根据需求描述, 自动生成网络请求、数据解析、页面布局等代码模板, 减少重复编码工作。

## 四、实验内容与步骤

### (一) 需求分析阶段

1. 定义数据结构: 设计新闻列表 JSON 数据模型, 包含 id、title、content、author、time 字段。
2. 确定 API 接口: 请求地址 `https://api.example.com/news`, 请求方式 GET, 返回 JSON 格式数据。
3. 明确功能需求: 实现新闻列表展示、网络数据请求、JSON 解析、下拉刷新, 扩展传感器集成。

### (二) Flutter 新闻列表开发 (2 学时)

1. 创建项目: 执行 `flutter create news_list` 命令初始化项目。
2. 添加依赖: 在 `pubspec.yaml` 中配置 `http`、`provider` 依赖。
3. 数据模型: 编写 `News` 类, 实现 `fromJson` 方法完成 JSON 解析。
4. 状态管理: 基于 `Provider` 创建 `NewsProvider`, 封装 `fetchNews` 网络请求方法。
5. 页面实现: 用 `ListView.builder` 构建列表, 集成 `RefreshIndicator` 实现下拉刷新。
6. 代码编写: 完成主函数、页面布局与数据绑定逻辑。

### (三) React Native 新闻列表开发 (2 学时)

1. 创建项目: 执行 `npx react-native-init NewsList` 初始化项目。
2. 安装依赖: 使用 `npm` 安装 `axios`、`refresh-control` 组件。
3. 状态管理: 用 `useState` 定义新闻数据与刷新状态, `useEffect` 初始化请求。
4. 列表实现: 通过 `FlatList` 渲染数据, 绑定 `keyExtractor` 与 `renderItem`。
5. 下拉刷新: 配置 `RefreshControl`, 实现 `onRefresh` 刷新逻辑。

#### 四) Uniapp 新闻列表开发 (2 学时)

1. 创建项目：在 HBuilderX 中新建 uni-app 项目。
2. 页面编写：在 index.vue 中用 v-for 循环渲染新闻列表。
3. 网络请求：使用 uni.request 调用 API，获取并处理返回数据。
4. 下拉刷新：配置 onPullDownRefresh 生命周期，请求完成后停止刷新。

#### (五) MAUI 新闻列表开发 (2 学时)

1. 创建项目：在 Visual Studio 中新建 .NET MAUI 项目。
2. 视图模型：基于 MVVM 模式编写 MainViewModel，定义 ObservableCollection 数据集合。
3. 网络请求：使用 HttpClient 发送 GET 请求，JsonSerializer 反序列化数据。
4. 数据绑定：将视图模型与页面绑定，实现列表自动更新。

#### (六) 传感器集成扩展

1. GPS 定位：在 Flutter 中集成 geolocator 插件，申请权限并获取经纬度。
2. 加速度计：使用 sensors\_plus 插件，监听三轴加速度数据并打印。

#### 七测试验证与报告编写

1. 功能测试：验证网络请求、数据解析、列表展示、下拉刷新是否正常。
2. 异常测试：模拟网络错误、空数据场景，测试容错能力。
3. 框架对比：从开发语言、性能、学习成本等维度对比四种框架。
4. 报告输出：整理实验数据，完成框架适用性分析报告。

### 五、实验结果与分析

#### (一) 功能实现结果

1. **Flutter**：列表展示流畅，下拉刷新响应快，Provider 状态管理稳定，无卡顿现象。
2. **React Native**：开发效率高，FlatList 性能良好，JS 桥接无明显延迟。
3. **Uniapp**：Vue 语法上手快，下拉刷新配置简单，适配多端一致性好。
4. **MAUI**：C#语法严谨，数据绑定便捷，原生兼容性优秀。

所有框架均成功完成\*\*API 请求、JSON 解析、列表展示、下拉刷新\*\*核心功能，传感器可正常获取定位与加速度数据。

(二) 框架对比分析

|       |         |              |         |       |
|-------|---------|--------------|---------|-------|
| 对比项   | Flutter | React Native | Uniapp  | MAUI  |
| 开发语言  | Dart    | JavaScript   | Vue     | C#    |
| 性能    | ★★★★★   | ★★★★☆        | ★★★☆☆   | ★★★★☆ |
| 学习成本  | 中等      | 低            | 低       | 中等    |
| 开发效率  | 较高      | 高            | 高       | 较高    |
| 跨平台能力 | 全平台     | 移动为主         | 小程序+App | 全平台   |
| 社区生态  | 完善      | 成熟           | 国内丰富    | 微软支持  |

(三) 问题与解决

1. 问题：Flutter 网络请求报错，提示 HTTPS 证书问题。

解决：添加网络权限，确认 API 接口地址正确。
2. 问题：React Native 下拉刷新不生效。

解决：正确导入 refresh-control 组件，绑定 refreshing 状态。
3. 问题：Uniapp 下拉刷新无响应。

解决：在 pages.json 配置 enablePullDownRefresh 为 true。
4. 问题：MAUI 数据解析失败。

解决：规范 NewsResponse 数据模型，匹配 JSON 结构。

六、实验总结

- 本次实验成功完成四种跨平台框架的新闻列表开发，掌握了网络请求、JSON 解析、下拉刷新等核心技能。
- 理解不同框架的技术特点：Flutter 性能最优，React Native 与 Uniapp 开发效率高，MAUI 适合.NET 技术栈开发者。
- 学会使用 AI 辅助编码工具，大幅提升代码编写速度，减少重复工作。
- 掌握移动设备传感器的集成方法，了解传感器在定位、运动监测等场景的应用。
- 通过测试验证与框架对比，明确各框架适用场景，为后续项目技术选型提供依据。

## 七、课后思考

1. 框架适用场景：Flutter 适合高性能全平台应用；React Native 适合前端开发者快速开发移动 App；Uniapp 适合小程序与 App 同步开发；MAUI 适合.NET 生态企业级应用。
2. 框架选择依据：根据项目需求、团队技术栈、性能要求、开发周期综合选择。
3. 传感器创新应用：结合 GPS 实现校园导航、签到；加速度计实现计步、摇一摇交互、设备姿态监测等功能。

## 八、实验心得

通过本次跨平台应用开发实验，我系统学习了 Flutter、React Native、Uniapp、MAUI 四种主流框架的开发流程，深刻体会到跨平台开发“一次编写、多端运行”的优势。在实验过程中，我不仅掌握了网络请求、数据解析、下拉刷新等核心技术，还学会了使用 AI 助工具提升开发效率。

同，通过对比不同框架的优缺点，我对跨平台技术选型有了更清晰的认知。传感器集成拓展让我了解到移动设备硬件的调用方式，为后续开发更丰富的移动应用打下基础。本次实验提升了我的实践能力与问题解决能力，收获颇丰。